

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	Bijoy Varghese	Reg. No.:	192411013
Section / Batch:	24 batch (3 rd yr)	Date:	22/07/2026

Instructions to Students

- Answer BOTH scenario-based questions. Each question carries 50 Marks. Total: 100 Marks.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions	Marks
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:			100 Marks

SECTION A – SCENARIO BASED ASSESSMENT

Code of Conduct

I Bijoy Varghese (192411013) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Bijoy Varghese

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Question No.	Problem Understanding & Context Analysis (25%)	Application of Concepts (30%)	Analytical & Decision-Making Skills (20%)	Solution Design & Approach (15%)	Clarity, Justification & Presentation (10%)	Total Marks (Each – 50)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

①
Q1) Student Records sorting (selection sort vs Bubble sort vs insertion sort).

Problem Understanding

The university stores the student mark in system which needs sort the records frequently. Therefore best is select sort or Bubble sort and insertion sort.

Application

Select sort:

* Find the minimum element in the unsorted portion and places it in correct position.

* Comparison remains high even if the data is nearly sorted.

Bubble sort:

* Compare adjacent elements and swaps them if required.

* With optimization, it stop early when no swap.

* Best case : $O(n)$

* Average / worst case : $O(n^2)$

Insertion Sort:

- * Insert each element into its correct position on the sorted part of array.
- * Perform very few shifts.

Analytical of Decision Making

<u>Algorithm</u>	<u>Best Case</u>	<u>Worst Case</u>	<u>Suitable</u>
Selection sort	$O(n^2)$	$O(n^2)$	No
Bubble sort	$O(n)$	$O(n^2)$	Good
Insertion sort	$O(n)$	$O(n^2)$	Best.

Solution:

Recommended: Insertion sort.

Due to.

- * Efficiently.

- * Easy to implement.

Clarity:

Insertion sort is the most suitable algorithm because the records are already sorted.

Q2) Linear Search in Hospital Database.

Problem Understanding:

The hospital store patient record is unsorted list because records are frequently added and uploaded.

Application

Linear Search checks each record one by one until the required patient records is found.

Best case : $O(1)$

Average : $O(n)$

Worst : $O(n)$

Analytical & Decision Making Skills

Since records are updated frequently keeping them sorted is difficult. Linear search is simple and works well in unsorted data.

Solution Design

Use Linear Search to search patient records directly without sorting, ensuring quick updates and simple implementation.

Clarity:

Linear search is suitable for this scenario because the database is frequently updated and sorted.

Q3) Linear Search vs Binary Search.

Problem Understanding:

The retail company stores transaction IDs in an unsorted dataset due to continuous updates. As data grows, searching slows.

Application:

Linear Search checks each transaction one by one.

Best case: $O(1)$

Average: $O(n)$

Worst: $O(n)$

If the data is sorted, Binary search can be used with $O(\log n)$

Analytical & Decision Making

For frequent searching, sorting once and using Binary search improves performance.

4) Binary Search in Sorted Student Records.

a) How Binary Search works.

Binary search works only on a sorted dataset. It compares the target element with the middle element. If the target is smaller, it searches in the left half, if larger, the right half.

Time Complexity.

Best Case: $O(1)$

Average Case = $O(\log n)$

Worst Case = $O(\log n)$

b) Importance of Maintaining Sorted Order.

Binary Search require the data to remain sorted. If the new records are inserted randomly and the order is disturbed, Binary search can't work properly. Therefore, Maintaining Sorted Order is so important.

Q5) Brute Force string Matching.

(a) Explain the working of Brute force string Matching.

Brute force string Matching compare the pattern with the text one character at a time. If a mismatch occurs, the pattern is shifted by one position and comparison start again.

(b) Time Complexity.

* Best case: $O(n)$

* Worst case: $O(n \times m)$

where

n = Length of text

m = Length of Pattern.

(c) Why Performance degrades:

Performance becomes slow for large documents because the algorithm compares the pattern at every possible position. When the pattern appears rarely or is absent, many unnecessary comparisons are made, increasing execution time.